Spotify Honk

Background Agents at Enterprise Scale

How Spotify built Honk, their background coding agent that has merged 1,500+ pull requests into production using Claude Code, constraint-based design, and 15 years of platform infrastructure.

ASDM Level 5 | Spotify | Music Streaming / Technology | June 2026

spotify | honk | level-5 | background-agents | claude-code | case-study

Key Metrics

Reported outcomes and operating signals from this case study.

1,500+

PRs merged by agents

AI agents have generated and merged over 1,500 pull requests into Spotify's production codebase.

REPORTED · Spotify Engineering Blog

650+

PRs merged monthly

The system merges approximately 650+ pull requests per month from agent-generated code.

REPORTED · Spotify Engineering Blog

60-90%

Time savings on migrations

Complex migrations show 60-90% time savings compared to manual code changes.

REPORTED · Spotify Engineering Blog

50%

Share of all PRs

Honk and related automation systems process approximately half of Spotify's pull requests.

REPORTED · Spotify Engineering Blog

CHALLENGE	APPROACH	OUTCOME
Spotify needed to automate large-scale code changes across thousands of components without introducing unpredictability or quality regressions.	Built Honk on Claude Code with intentional constraints (3 tools only), deterministic verification, LLM-as-Judge layer, and 15 years of platform infrastructure including Backstage.	1,500+ merged PRs, 650+ monthly, with engineers able to ship code from their phones during morning commutes.

ASDM Level 5: Agentic Engineering

Spotify's Honk is the most thoroughly documented enterprise deployment of background coding agents. The system has merged over 1,500 pull requests into production, operates at 650+ PRs per month, and represents a fundamentally different approach to AI-assisted development: **constraint-based design over capability maximization**.

This case study demonstrates ASDM Level 5: AI writes most code with human review as the final gate. But more importantly, it shows what infrastructure is required to make background agents reliable at scale.

The Core Problem

Spotify maintains thousands of software components owned by hundreds of engineering teams. Large-scale changes—dependency updates, framework migrations, API deprecations—require coordinated modifications across this ecosystem.

Traditional approaches had hit their limits:

- **Deterministic scripts** became unmaintainable (one Maven dependency updater exceeded 20,000 lines of code)

Manual coordination across teams created months-long migration timelines
AI-assisted tools helped individual developers but didn't scale to organization-wide changes

Spotify needed agents that could work **in the background**—kicked off once and running to completion without requiring an engineer to be actively engaged.

How Honk Works

Runtime Architecture

Honk runs Claude Code in sandboxed Kubernetes Jobs. Each job processes a single codebase transformation, creating a branch, making changes, and submitting a pull request only after all validation passes.

The critical design decision: **intentional constraint**.

Rather than exposing extensive tool access, Spotify limits agents to three tools:

Tool	Purpose
Verify	Run deterministic verifiers (lint, format, tests)
Git	Allowlisted operations for branch/commit/push
Bash	Allowlisted commands including ripgrep for search

This constraint is deliberate. In Spotify's experience, "the more tools you have, the more dimensions of unpredictability you introduce."

Two-Tier Quality Control

Honk implements a two-tier verification system:

Tier 1: Deterministic Verifiers

Deterministic tools activate based on file content:

- Maven verifier triggers on `pom.xml` changes
- Python verifier triggers on `.py` file changes
- TypeScript verifier triggers on `.ts` file changes

These verifiers run independently of LLM behavior. The agent cannot bypass them.

Tier 2: LLM-as-Judge

On top of deterministic checks, an LLM evaluates whether changes stay in scope. The judge compares diffs against the original prompt to detect scope creep.

Spotify's data: "Out of thousands of agent sessions, the judge vetoes about a quarter of them. When that happens, the agent is able to course correct half the time."

This means roughly 12% of sessions require the judge to prevent out-of-scope changes—a significant quality filter.

Context Engineering Over Tool Access

Spotify made a deliberate architectural choice: **context over skills**.

Rather than exposing MCP tools for reading schemas, accessing databases, or querying APIs, developers pre-populate prompts with all necessary context. The context file becomes the guardrail.

For complex migrations, teams create detailed mapping tables:

```
## Migration Mapping

| Old Pattern | New Pattern | Notes |
|-------------|-------------|-------|
| `BigQueryRunner.execute()` | `BigQueryClient.run()` | Direct replacemer
| `LegacyConfig.get()` | `ConfigService.fetch()` | Requires context injec
| `ScioJob.run()` | DO NOT MIGRATE | Too much variability |
```

The “DO NOT MIGRATE” entries are critical—they tell the agent where not to attempt changes.

Real-World Results

Dataset Migration Case Study

Spotify migrated approximately 1,800 direct downstream data pipelines across three frameworks:

BigQuery Runner: Direct SQL execution
dbt: Transformation framework
Scio: Scala data processing (excluded from automation)

Results:

240 automated pull requests generated
10 engineering weeks saved compared to manual effort
60-90% time reduction on complex migrations

Scio was excluded because “the variability in how Scio jobs were structured made automation unreliable.” Knowing when not to automate is as important as automation itself.

Daily Operations

Engineers can now:

- Kick off migration tasks from Slack
- Submit change requests via Git prompts
- Ship code from their phones during morning commutes

The agent handles the implementation; humans handle the intent.

What Made Honk Possible

Honk's success wasn't built on LLMs—it was built on 15 years of platform engineering.

Backstage: The Developer Portal

Spotify's Backstage catalogs thousands of software components with:

- **Ownership information**: Who maintains each component
- **Dependency graphs**: What depends on what
- **Service metadata**: Build systems, deployment targets, configuration

The principle: "You can't safely automate what you don't understand." Backstage provides the understanding.

Fleet Management

Before Honk, Spotify had already built Fleet—a scaled automation pipeline running Docker-based transformations across thousands of repositories via Kubernetes.

Fleet already processed half of Spotify's pull requests before AI was involved. Honk extended Fleet's capabilities with LLM-based reasoning.

Java Bill of Materials (BOM)

96% adoption of a centralized dependency management system ensured consistency across the engineering organization. When the BOM updates, agents know exactly what versions to target.

Architectural Lessons

1. Constraints Enable Predictability

Honk succeeds by limiting what agents can do, not maximizing their capabilities:

- Three tools instead of dozens
- Pre-populated context instead of dynamic tool access
- Explicit exclusion lists for complex cases

2. Verification is Infrastructure, Not Feature

Deterministic verification runs independently of LLM behavior. Agents cannot modify the verification pipeline. This separation is critical for trust.

3. Context Engineering Replaces Prompt Engineering

The quality of agent output correlates with context quality, not prompt sophistication. Detailed mapping tables, exclusion lists, and concrete examples drive reliability.

4. Feedback Loops Enable Self-Correction

When the LLM judge vetoes a change, agents can often self-correct. This iterative capability reduces human intervention while maintaining quality.

5. Infrastructure First, AI Second

Backstage, Fleet, and the BOM existed before Honk. The AI layer amplified existing capabilities rather than creating them from scratch.

ASDM Classification

Under the Software Delivery Autonomy Model, Spotify Honk operates at **Level 5: AI Writes Most Code**.

Criterion	Honk Implementation
Code generation	Claude Code generates changes
Human review	Required before merge
Scope	Organization-wide migrations
Validation	Two-tier: deterministic + LLM-as-Judge

Honk does not reach Level 6 because human review remains in the loop. Pull requests require human approval before merge. The system generates and validates code, but humans make the final trust decision.

Implications for Enterprise Adoption

Prerequisites

Organizations considering background agents should assess:

Component catalog: Do you know what you have and who owns it?
Build system consistency: Can you reliably build and test any component?
Dependency management: Are versions standardized across the organization?

Automation infrastructure: Do you have pipeline capacity for high-volume jobs?

Without these foundations, background agents become unpredictable.

Constraint-Based Design

The Honk model inverts typical AI integration thinking:

Typical approach: "What capabilities can we give the agent?"
Honk approach: "What is the minimum access needed for reliable output?"

Fewer tools means fewer failure modes.

Organizational Change

Background agents shift engineering work:

From: Writing code, reviewing code
To: Designing contexts, maintaining catalogs, defining constraints

The skills that matter change when agents handle implementation.

Conclusion

Spotify Honk demonstrates that enterprise-scale background agents are possible—with the right infrastructure. The 1,500+ merged PRs prove the model works. The 15 years of platform engineering prove the model has prerequisites.

For organizations evaluating background agents, Honk provides a clear message: **infrastructure enables AI, not the reverse**. Backstage, Fleet, and the BOM created the conditions where Honk could succeed. Without that foundation, the same AI models would produce unpredictable results.

The constraint-based design philosophy—three tools, pre-populated context, explicit exclusions—offers a template for reliable agent deployment. More capability isn't

always better. Sometimes less is the path to production.

Sources

- 1,500+ PRs Later: Spotify's Journey with Our Background Coding Agent (Honk, Part 1)
- Background Coding Agents: Context Engineering (Honk, Part 2)
- Background Coding Agents: Predictable Results Through Strong Feedback Loops (Honk, Part 3)
- Background Coding Agents: Supercharging Downstream Consumer Dataset Migrations (Honk, Part 4)
- Coding Is No Longer the Constraint: Scaling Developer Experience at Spotify